Zbuduj kalkulator sprawdzianu

Masz gotowy wygląd strony zapisany w HTML z lekcji 2. Twoim zadaniem jest samodzielnie dopisać JavaScript, który ożywi dwa istniejące przyciski. Pracujesz cały czas nad tym samym projektem.

Efekt po dwóch lekcjach
Wpisujesz zdobyte i maksymalne punkty. Aplikacja sprawdza dane, oblicza procent i informuje o zaliczeniu od 40%. Przycisk „Wyczyść” usuwa dane oraz przywraca początkowy wygląd komunikatu.

Przygotuj miejsce do pracy

1. Utwórz własny folder roboczy.
Skopiuj do niego tylko **index.html z lesson2**. Utwórz obok pusty plik **script.js** i pusty **styles.css**. Nie kopiuj gotowego JavaScriptu z katalogów lekcji.

2. Sprawdź połączenie plików.
W otrzymanym HTML jest już znacznik poniżej. Nazwa pliku musi być dokładnie taka sama; script.js nie może mieć ukrytego rozszerzenia .txt.

```
<script src="script.js" defer></script>
```

defer oznacza tutaj, że zewnętrzny skrypt wykona się po odczytaniu HTML. Dzięki temu JavaScript może odnaleźć pola i przyciski. Nie zmieniaj HTML, aby zastępować go strukturą budowaną w JS.

3. Uruchom stronę i konsolę.
Otwórz index.html w przeglądarce. Otwórz narzędzia deweloperskie, np. przez menu „Zbadaj / Inspect”, i zakładkę **Console**. Na Macu w Chrome skrót to Option + Command + J. Po zmianie JS zapisz plik i odśwież stronę.

Bootstrap pobiera wygląd z internetu. Na tych lekcjach nie dopisujesz własnego CSS, nie używasz bibliotek JS ani zapisu danych.

Jak korzystać z tej instrukcji

Najpierw przeczytaj krótką teorię, potem wykonaj zadania. Przykłady pokazują pojedyncze narzędzia, a nie pełne rozwiązanie kalkulatora. Gdy coś nie działa, sprawdź konsolę i tabelę testów.

Co już masz na stronie?

Element	Identyfikator / działanie
Zdobyte punkty	id="points"
Maksymalna liczba punktów	id="maxPoints"
Akapit z wynikiem	id="result"
Przycisk „Oblicz wynik”	onclick="calculateResult('')"
Przycisk „Wyczyść”	onclick="clearResult('')"

id jest nazwą konkretnego elementu HTML. JavaScript odnajduje element po tej nazwie. Wielkość liter ma znaczenie: **maxPoints** to inna nazwa niż maxpoints.

Dwie funkcje do napisania

```
function calculateResult() {
  // Tutaj napisz pobranie danych i obliczenia.
}

function clearResult() {
  // Tutaj napisz czyszczenie pól i komunikatu.
}
```

To szkielet, od którego możesz zacząć plik script.js. **function** definiuje funkcję, czyli nazwany zestaw instrukcji. Nawiasy **()** służą do przekazywania danych, a klamry **{ }** obejmują instrukcje wykonywane wewnątrz funkcji.

Sama definicja funkcji jeszcze jej nie uruchamia. Robi to przycisk: atrybut **onclick** wywołuje funkcję po kliknięciu. Nazwy muszą się zgadzać znak po znaku.

Dlaczego w HTML jest calculateResult('')?
Przycisk przekazuje pusty tekst jako argument. Na lekcjach 1 i 2 możesz zdefiniować funkcję bez parametrów, tak jak powyżej. Przekazany argument nie będzie używany. Nie musisz zmieniać HTML.

Pola do obliczeń

Do odczytywania i czyszczenia pól użyj właściwości **value**. Wynik wyświetlaj w istniejącym akapicie result przez **textContent**.

Pola mają type="number", min="0" i step="any". Służą do wpisywania liczb, również ułamków. Po kliknięciu przycisku funkcja odczyta ich aktualną zawartość. Sprawdzanie poprawności danych dopiszesz w lekcji 2.

Jak odczytać dane i pokazać wynik?

1. const i odwołanie do elementu

```
const field = document.getElementById('quantity');
```

document oznacza dokument strony. **getElementById()** wyszukuje element o podanym id. **const** tworzy zmienną, której nie można później przypisać nowej wartości. Tutaj field przechowuje odwołanie do elementu, a nie wpisaną liczbę. W twoim HTML nie ma pola quantity - to przykład składni.

2. value oraz Number()

```
const text = field.value;
const quantity = Number(text);
```

.value odczytuje zawartość pola jako tekst, nawet przy type="number". **Number()** zamienia ten tekst na liczbę. Odczytuj .value dopiero po kliknięciu przycisku, wewnątrz funkcji - wtedy użyjesz aktualnych danych.

```
console.log('12' + 3);         // 123 (tekst)
console.log(Number('12') + 3); // 15 (liczba)
```

3. Obliczenia i przypisanie

Znak **=** przypisuje wartość do zmiennej. Operatory: **+** dodaje liczby lub łączy tekst, **-** odejmuje, ***** mnoży, **/** dzieli. Nawiasy pozwalają określić kolejność działań. Procent obliczysz ze wzoru:

```
procent = (zdobyte punkty / maksymalne punkty) * 100
```

To zapis wzoru, nie gotowa linia JS: nazwy zmiennych nie mogą zawierać spacji. Dla 18 punktów z 30 otrzymasz 60%. Na końcu wyświetlanego tekstu dopisz znak %.

4. textContent i console.log()

```
const message = document.getElementById('notice');
message.textContent = 'Liczba sztuk: ' + quantity;
console.log(quantity);
```

.textContent zmienia zwykły tekst istniejącego elementu. **console.log()** pokazuje wartość w konsoli, nie na stronie. Użyj obu sposobów, żeby zobaczyć wynik i łatwiej znaleźć błędy. Nie używaj innerHTML ani tworzenia nowych znaczników.

Oblicz procent zdobytych punktów

Cel: po kliknięciu „Oblicz wynik” użytkownik widzi procent, a po „Wyczyść” może zacząć od nowa. W tej lekcji testuj poprawne dane; sprawdzanie błędów jest zadaniem lekcji 2.

Część A. Funkcja calculateResult()

1. Znajdź oba pola.
Wewnątrz funkcji zapisz w dwóch zmiennych const odwołania do elementów points i maxPoints. Nie pomyl id pola z nazwą wymyślonej przez siebie zmiennej.

2. Odczytaj i zamień wartości.
Z każdego pola pobierz .value i zamień tekst na liczbę za pomocą Number(). Zachowaj osobno odwołania do pól i osobno liczby; przydadzą się w następnej lekcji.

3. Oblicz procent.
Użyj wzoru z poprzedniej strony. Zapisz wynik w kolejnej zmiennej. Nie odejmuj punktów i nie mnoż przed podzieleniem przez maksimum, jeśli zmieniłoby to wzór.

4. Pokaż rezultat w dwóch miejscach.
Wypisz liczbę w konsoli. Odnajdź istniejący akapit result i ustaw jego textContent tak, aby dla 35/50 pojawiło się **„Wynik: 70%”**.

Część B. Funkcja clearResult()

Przypisanie pustego tekstu **''** usuwa zawartość. Pojedyncza linia dla przykładowego pola wygląda tak:

```
document.getElementById('quantity').value = '';
```

W swojej funkcji wyczyść w ten sposób **points** i **maxPoints**. Ustaw textContent akapitu result na pusty tekst albo na **„Wpisz punkty i oblicz wynik.”**. W konsoli pokaż komunikat „Pola wyczyszczone”.

Test przed zakończeniem lekcji

Punkty	Maksimum	Wynik
35	50	70%
10	20	50%
0	20	0%
20	20	100%
7,5	10	75%

Wpisz liczby ułamkowe separatorem akceptowanym przez przeglądarkę (kropką lub przecinkiem). Testuj też: Oblicz → Wyczyść → wpisz inne dane → Oblicz. Funkcje mają działać wiele razy bez odświeżania.

Warunki: kiedy program ma się zatrzymać?

1. if oraz else

if wykonuje kod, gdy warunek jest prawdziwy. **else** opisuje drugi przypadek. Przykład dotyczy magazynu, nie twojego kalkulatora:

```
const stock = 8;
if (stock >= 5) {
  console.log('Wystarczający zapas');
} else {
  console.log('Mały zapas');
}
```

Operator	Znaczenie	Przykład wyniku
>=	Większe lub równe	5 >= 5 daje true
>	Większe	5 > 5 daje false
<	Mniejsze	3 < 5 daje true
<=	Mniejsze lub równe	0 <= 0 daje true
===	Ściśle równe	'' === '' daje true

true oznacza prawdę, **false** fałsz. Nie pomyl **=** (przypisanie) z **===** (porównanie). Tekst wpisuj w cudzysłowie lub apostrofach, liczby bez nich.

2. return bez wartości

```
function checkStock() {
  const stock = -2;
  if (stock < 0) {
    console.log('Niepoprawna liczba sztuk');
    return;
  }
  console.log('Dane można dalej przetwarzać');
}
```

return; kończy bieżące wywołanie funkcji. Po wykryciu błędu pokaż komunikat i zakończ funkcję, aby późniejsze obliczenia nie nadpisały błędu. W tej lekcji wystarczą oddzielne if - nie musisz jeszcze łączyć warunków operatorami logicznymi.

Puste pole a zero
Number('') daje 0. Dlatego sprawdzaj, czy surowe pole.value jest puste, zanim uznasz przekonwertowaną liczbę za poprawną. Wynik 0 punktów przy dodatnim maksimum jest prawidłowy; pustego pola nie wolno traktować jak świadomie wpisanego zera.

Sprawdź dane i dodaj zaliczenie

Start: samodzielna rozgrzewka

Na początku zajęć, bez kopiowania rozwiązania, popraw clearResult(), jeśli jeszcze nie czyści obu pól i komunikatu. Następnie pokaż działające obliczenie 18/30 = 60%. Dopiero po tej próbie przejdź do warunków.

Część A. Walidacja przed obliczeniem

Rozbuduj istniejącą calculateResult(). Najpierw pobierz pola, liczby i akapit result. Potem wykonuj kontrole w poniższej kolejności. W każdym przypadku błędu ustaw komunikat w result i użyj return. Dopiero po wszystkich kontrolach oblicz procent.

Kolejność	Warunek do wykrycia	Komunikat
1	Puste pole points	Podaj zdobyte punkty.
2	Puste pole maxPoints	Podaj maksymalną liczbę punktów.
3	Maksimum mniejsze lub równe 0	Maksymalna liczba punktów musi być większa od zera.
4	Zdobyte punkty mniejsze od 0	Punkty nie mogą być ujemne.
5	Zdobyte punkty większe od maksimum	Zdobyte punkty nie mogą przekraczać maksimum.

Puste pola porównuj z '' przez ===. W pozostałych kontrolach używaj liczb po konwersji. Sprawdzenie maksimum przed dzieleniem zapobiega nieprawidłowym wynikom, np. Infinity albo NaN.

Część B. Decyzja o zaliczeniu

1. Po walidacji oblicz procent.
Zachowaj wzór z lekcji 1 oraz wypisanie wyniku w konsoli.

2. Użyj if/else.
Jeśli procent jest większy lub równy 40, pokaż wynik i „Sprawdzian zaliczony!”. W przeciwnym razie pokaż wynik i „Sprawdzian niezaliczony!”. Wykorzystaj jedną instrukcję if/else.

3. Sprawdź wartość graniczną.
Dla 40% sprawdzian musi być zaliczony. Nie używaj samego > 40. Dla 39% ma być niezaliczony. Na razie nie przeliczaj procentów na oceny 1-6.

Przykładowy tekst końcowy: **„Wynik: 80% - Sprawdzian zaliczony!”**. Równoważna interpunkcja nie zmienia poprawności zadania.

Użyj istniejących klas Bootstrapa

HTML ma już akapit result. Nie twórz kolejnego elementu i nie dopisuj stylów w CSS. Zmieniaj jego tekst oraz zestaw klas Bootstrap zależnie od sytuacji.

Co robi className?

```
const notice = document.getElementById('notice');
notice.className = 'alert alert-success mt-3 mb-0';
```

.className ustawia cały atrybut class elementu. W tym przykładzie **alert** nadaje wygląd komunikatu, **alert-success** zielony wariant, **mt-3** odstęp nad nim, a **mb-0** brak dolnego marginesu. notice jest przykładem; w projekcie pracujesz z result.

Sytuacja	Wartość className dla result
Zaliczenie	alert alert-success mt-3 mb-0
Brak zaliczenia lub błąd	alert alert-danger mt-3 mb-0
Po wyczyszczeniu	alert alert-light border mt-3 mb-0

Ponieważ className zastępuje cały zestaw klas, nie wpisuj wyłącznie alert-success. Przy kolejnym obliczeniu ustaw kolor ponownie - wynik po wcześniejszym błędzie musi móc stać się zielony.

Rozbuduj clearResult()

Oprócz usunięcia zawartości pól i komunikatu przywróć neutralny zestaw klas z tabeli. Dzięki temu po wyczyszczeniu nie zostanie czerwony albo zielony komunikat. Zachowaj log w konsoli.

Testy prawidłowych wyników

Punkty / maksimum	Oczekiwane zachowanie
40 / 50	80%, zaliczony, zielony komunikat
10 / 50	20%, niezaliczony, czerwony komunikat
39 / 100	39%, niezaliczony
40 / 100	40%, zaliczony
41 / 100	41%, zaliczony
0 / 20	0%, niezaliczony - to nie jest błąd danych
20 / 20	100%, zaliczony

Nie zaokrąglaj wyniku przed porównaniem z progiem. Na tych lekcjach możesz pokazywać pełną liczbę zwracaną przez obliczenie. Formatowanie do wybranej liczby miejsc po przecinku to zadanie dodatkowe.

FINAŁ / SAMODZIELNE SPRAWDZENIE

Przetestuj błędy i oddaj projekt

Wpisane dane	Co ma się wydarzyć
Puste points, maxPoints = 20	Komunikat o brakujących zdobytych punktach
points = 10, puste maxPoints	Komunikat o brakującym maksimum
Oba pola puste	Pierwszy komunikat: podaj zdobyte punkty
10 / 0 oraz 10 / -5	Błąd: maksimum musi być większe od zera
-1 / 20	Błąd: punkty nie mogą być ujemne
25 / 20	Błąd: punkty przekraczają maksimum

Sprawdź kolejne kliknięcia, nie tylko jeden wynik

1. Błąd → poprawny wynik.
Wpisz 60/50, kliknij Oblicz, potem popraw punkty na 40 i kliknij ponownie. Błąd ma zostać zastąpiony przez zielony wynik 80%.

2. Zaliczenie → brak zaliczenia.
Po 40/50 sprawdź 10/50. Tekst i kolor muszą zmienić się razem.

3. Wyczyść → policz ponownie.
Wyczyść po błędzie. Oba pola mają być puste, komunikat pusty lub początkowy, wygląd neutralny. Potem 5/10 ma dać zaliczone 50%.

Gdy coś nie działa

Objaw	Sprawdź
calculateResult is not defined	Nazwę funkcji, nazwę script.js, zapis pliku i odświeżenie strony.
Cannot read properties of null	Czy id jest dokładnie points, maxPoints lub result.
W konsoli jest wynik, na stronie nie	Czy ustawiasz textContent akapitu result.
Błąd znika i pojawia się procent	Czy po komunikacie błędu znajduje się return.

Co oddajesz?

Folder z otrzymanym index.html i własnym script.js (oraz pustym styles.css, jeśli pozostawiasz jego podłączenie). JS zawiera dwie funkcje: calculateResult() i clearResult(). Każda pozycja z tabel testów działa. Przygotuj się do pokazania, które linie pobierają dane, obliczają wynik i zatrzymują funkcję po błędzie.

Dokumentacja do dalszej nauki: MDN: getElementById · Number · textContent · if/else · return · Bootstrap: alerty.